Knight's Tour

Pavlov Aleksandr

29.11.2025

1 Question 1 and 2

I tried several ways to encode the problem into SAT to find a fast one.

1. **Per-cell-per-turn encoding.** First, I tried the simplest way: create a variable for every square on the board for every move. I added constraints so that for each move, exactly one square is visited, and for each square, it is visited exactly once, using pairwise negation (or pairwise constraints). But this method took over a second on my computer for an 8x8 board.

2. **Rows and columns encoding.** I also tried to encode the row and column for each move, instead of the individual squares. This reduced the number of variables, but it greatly increased the number of clauses, and the solving process became extremely slow.

3. **Final encoding.** I noticed that a knight always moves to a square of the opposite color. This means that if you know its starting position, you can calculate which color it will be on for every single move. Using this, I eliminated half of the square variables that were impossible. This made the solver 10 times faster for the 8x8 board compared to the first method. The final solution uses the following constraints:

   - **One move per turn.**
     - **At least one:** For each turn t, at least one cell variable must be true.
     - **At most one:** For each turn t, if the knight is on one cell, it cannot be on any other. This is enforced with pairwise negation for all pairs of cells for that turn.
   - **One visit per cell.**
     - **At least once:** For each non-start cell, it must be visited at least once on a turn of the correct parity.
     - **At most once:** For each cell, if it is visited on one turn, it cannot be visited on another. This is enforced with pairwise negation over all allowed turns for that cell.
   - **Knight move rules.**
     - **First move:** The first move is constrained to be one of the valid knight moves from the start position.
     - **Next moves:** For every cell and turn t (except the last turn), if the knight is on that cell at t, then at turn $t + 1$ it must be on one of its valid knight target cells.

2 Question 3

To find all solutions, I repeatedly found a solution, added model of that solution to the constraints, and continued until all solutions were found.

3 Question 4

After finding all solutions, I compared them with each other using reflections to check for symmetry and remove symmetric solutions.

4 Question 5

Take one random solution and use it as a set of constraint cells. Then remove these constraint cells one by one. After removing a cell, check whether new solutions appear. If new solutions appear, keep that cell in the constraint set and continue.